```
<%= image_tag("/assets/dave.png", class: "bevel", size: "80x120") %>
<%= image_tag("/assets/andy.png", class: "bevel",
              width: "80", height: "120") %>
```

If you don't give an `:alt` option, Rails synthesizes one for you using the image's filename. If the image path doesn't start with a / character, Rails assumes that it lives under the `app/assets/images` directory.

You can make images into links by combining `link_to` and `image_tag`:

```
<%= link_to(image_tag("delete.png", size: "50x22"),
            product_path(@product),
            data: { confirm: "Are you sure?" },
            method: :delete)
%>
```

The `mail_to` helper creates a `mailto:` hyperlink that, when clicked, normally loads the client's email application. It takes an email address, the name of the link, and a set of HTML options. Within these options, you can also use `:bcc`, `:cc`, `:body`, and `:subject` to initialize the corresponding email fields. Finally, the magic option `encode: "javascript"` uses client-side JavaScript to obscure the generated link, making it harder for spiders to harvest email addresses from your site. Unfortunately, it also means your users won't see the email link if they have JavaScript disabled in their browsers.

```
<%= mail_to("support@pragprog.com", "Contact Support",
            subject: "Support question from #{@user.name}",
            encode:  "javascript") %>
```

As a weaker form of obfuscation, you can use the `:replace_at` and `:replace_dot` options to replace the at sign and dots in the displayed name with other strings. This is unlikely to fool harvesters.

The `AssetTagHelper` module also includes helpers that make it easy to link to style sheets and JavaScript code from your pages and to create autodiscovery Atom feed links. We created links in the layouts for the Depot